2.6 Cloning Objects

> Supplemental topic: object cloning is not covered in Chapters 17–18, but it follows naturally from the discussion of object identity in 2.5.

Assigning an object to another variable does not create a second object:

```php
$original = new Person('Alice');
$alias = $original;

$alias->setDisplayName('Bob');

echo $original->getDisplayName(); // Bob
echo $original === $alias ? 'same' : 'different'; // same
```

Both variables identify the same object. Use `clone` when you need a distinct object with the same initial property values:

```php
$original = new Person('Alice');
$copy = clone $original;

$copy->setDisplayName('Bob');

echo $original->getDisplayName(); // Alice
echo $copy->getDisplayName();     // Bob
echo $original === $copy ? 'same' : 'different'; // different
```

The constructor does not run during cloning. PHP first copies the object's properties and then invokes `__clone()` on the new object, if that method exists.

The default clone is shallow

Scalar property values are copied, but properties that identify other objects still identify the same nested objects:

```php
class Profile
{
    public function __construct(public string $bio)
    {
    }
}

class Member
{
    public function __construct(
```

```
        public string $displayName,
        public Profile $profile,
    ) {
    }
}

$original = new Member('Alice', new Profile('Hello!'));
$copy = clone $original;
$copy->profile->bio = 'Updated through the clone';

echo $original->profile->bio; // Updated through the clone
```

The two `Member` objects are separate, but both `profile` properties still identify the same `Profile` object. This is a **shallow copy**.

Customize the clone with `__clone()`

Clone nested mutable objects inside `__clone()` when the copy must be independent:

```
class Member
{
    private static int $nextId = 1;

    public int $id;

    public function __construct(
        public string $displayName,
        public Profile $profile,
    ) {
        $this->id = self::$nextId++;
    }

    public function __clone(): void
    {
        $this->id = self::$nextId++;
        $this->profile = clone $this->profile;
    }
}
```

`__clone()` runs on the new object, so `$this` refers to the clone. This implementation assigns a new identity and clones the nested profile. A useful `__clone()` might also clear workflow state that should not carry into a copy.

Do not automatically clone every nested object. Some related objects should remain shared. For example, cloning a `Post` might create an editable copy of the post while deliberately retaining the same author object.

Cloning readonly properties in PHP 8.3+

PHP 8.3 permits a `readonly` property to be reinitialized from the protected scope of `__clone()`. This makes an independent nested clone possible:

```
class Member
{
    public function __construct(public readonly Profile $profile)
    {
    }

    public function __clone(): void
    {
        $this->profile = clone $this->profile;
    }
}
```

The reinitialization is allowed only as part of cloning; the property remains readonly afterward.

For the full language behavior, see the PHP manual pages for object cloning and readonly properties.